

Звіт

з лабораторної роботи №3

Стегня Віктора ПМі-44

Тема

Реалізація хвильових алгоритмів відлуння та деревоподібної хвилі в симуляторі розподіленої системи.

Опис реалізації

Основна логіка була реалізована у файлі `scheduler/implementation/node.py`. Для лабораторної роботи в симулятор додано два режими роботи:

- `echo` — централізований хвильовий алгоритм відлуння для зв'язного неорієнтованого графа;
- `tree` — децентралізований хвильовий алгоритм для деревоподібної мережі, який ініціюється листками дерева.

Для зберігання стану одного запуску реалізовано окремі структури:

- `EchoState` — містить ознаки ініціатора, батька, список очікуваних відповідей, дочірні вершини дерева відлуння та стан завершення;

- `TreeWaveState` — зберігає множину сусідів, від яких уже отримано хвильове повідомлення, батьківський вузол, множину відомих рішень і стан завершення хвилі.

Для алгоритму відлуння реалізовано такі типи повідомлень:

- `New` — зовнішній запуск алгоритму;
- `ECHO_WAVE` — пряме хвильове повідомлення;
- `ECHO_REPLY` — зворотне повідомлення до батьківського вузла.

Логіка алгоритму відлуння така:

- 1 Ініціатор розсилає хвилю всім своїм сусідам.
- 2 Кожен вузол при першому отриманні хвилі запам'ятовує відправника як батька.
- 3 Потім вузол пересилає хвилю всім сусідам, крім батька.
- 4 Після отримання всіх зворотних відповідей вузол надсилає одну відповідь своєму батьку.
- 5 Ініціатор після отримання відповідей від усіх сусідів приймає рішення і завершує хвилю.

Для деревоподібної хвилі реалізовано такі типи повідомлень:

- `New` — зовнішній запуск алгоритму;

- `TREE_WAVE` — хвильове повідомлення в напрямку до центру дерева;
- `TREE_NOTIFY` — повідомлення про один із двох вузлів, що прийняли рішення.

Логіка алгоритму для дерева така:

- 1 Кожний лист дерева надсилає хвильове повідомлення своєму єдиному сусіду.
- 2 Внутрішній вузол чекає, доки не отримає хвильові повідомлення від усіх сусідів, крім одного.
- 3 Останній сусід розпізнається як батько, і вузол відправляє хвилю саме йому.
- 4 Коли вузол отримує хвильове повідомлення від свого батька, він стає одним із двох вузлів прийняття рішення.
- 5 Після цього по дереву розсилаються повідомлення `TREE_NOTIFY`, доки кожен вузол не дізнається рівно про два рішення.

Зміни в симуляторі

- у `scheduler/settings/network_settings.py` додано параметр `WAVE_ALGORITHM` для перемикання між режимами `echo` і `tree`.
- у `scheduler/implementation/current_network.py`

додано дві топології мережі: загальний неорієнтований граф для алгоритму відлуння та дерево для деревоподібної хвилі.

- у

`scheduler/implementation/graph_utils.py`
реалізовано перевірки `is_neighborhood_correct`,
`is_undirected` та `is_tree`.

- у

`scheduler/core/external_request_generator.py`
зовнішні запити адаптовано до обох алгоритмів: для `echo` стартує один ініціатор, а для `tree` одночасно активуються всі листки дерева.

- у

`scheduler/abstract/abstract_network.py` та `scheduler/core/observer.py` додано визначення завершення симуляції, коли в мережі більше немає доступних дій і нові зовнішні запити не генеруються.

- Результати завершених запусків записуються у файл `test_results/lab3_results.txt`.

Результати роботи

Після реалізації було виконано перевірку роботи симулятора для обох алгоритмів.

- Для алгоритму відлуння підтверджено, що тільки один вузол-ініціатор приймає рішення після отримання всіх відповідей від дерева відлуння.

- Для деревоподібної хвилі підтверджено, що рішення приймають рівно два центральні вузли, після чого інформація про обидва рішення поширюється на всю мережу.
- В обох випадках симулятор коректно завершує виконання без зависання після вичерпання всіх повідомлень.
- Окремо перевірено коректність допоміжних функцій для перевірки графа та інтеграційні сценарії для обох алгоритмів.

Для автоматичної перевірки було додано тести:

- `tests/test_graph_utils.py` — перевірка коректності подання графа, неорієнтованості та властивості дерева;
- `tests/test_wave_algorithms.py` — перевірка завершення алгоритму відлуння та перевірка того, що в дереві існує рівно два вузли прийняття рішення.

Також виконано запуски:

- `python3 -m unittest discover tests;`
- `python3 run_simulation.py.`

Висновки

У ході лабораторної роботи було реалізовано два різних хвильових алгоритми на базі одного симулятора розподіленої системи. Алгоритм відлуння показує централізовану схему поширення та згортання хвилі, де рішення приймає ініціатор. Деревоподібний алгоритм демонструє децентралізований підхід для мережі без циклів, у якому хвиля сходиться до центру дерева, а рішення приймають два центральні вузли.

У результаті симулятор було розширено підтримкою двох режимів роботи, перевіркою властивостей топології та автоматичними тестами. Отримана реалізація відповідає теоретичному опису хвильових алгоритмів і придатна для подальших експериментів із розподіленими обчисленнями.